Socho ek school mein **Teacher** hain.

Teacher ke paas kuch common abilities hain:

- Padhana
- Attendance lena
- Students ko guide karna

Ab school mein alag-alag teachers hain:

- 👨‍🏫 Math Teacher
- 👩‍🔬 Science Teacher
- 👩‍🏫 English Teacher

Ye sab **Teacher** hain, isliye in sabko dobara padhana aur attendance lena nahi sikhana padta. Ye abilities unhe **Teacher** se mil jaati hain.

Lekin har teacher ki apni ek special ability bhi hoti hai:

- Math Teacher → Maths padhata hai.
- Science Teacher → Experiments karvata hai.
- English Teacher → Grammar aur Speaking sikhata hai.

Yaani:

- **Common abilities** sabko Teacher se mili.
- **Special abilities** har teacher ki apni hain.

Isi concept ko programming mein **Inheritance** kehte hain.

By Ved Vyas

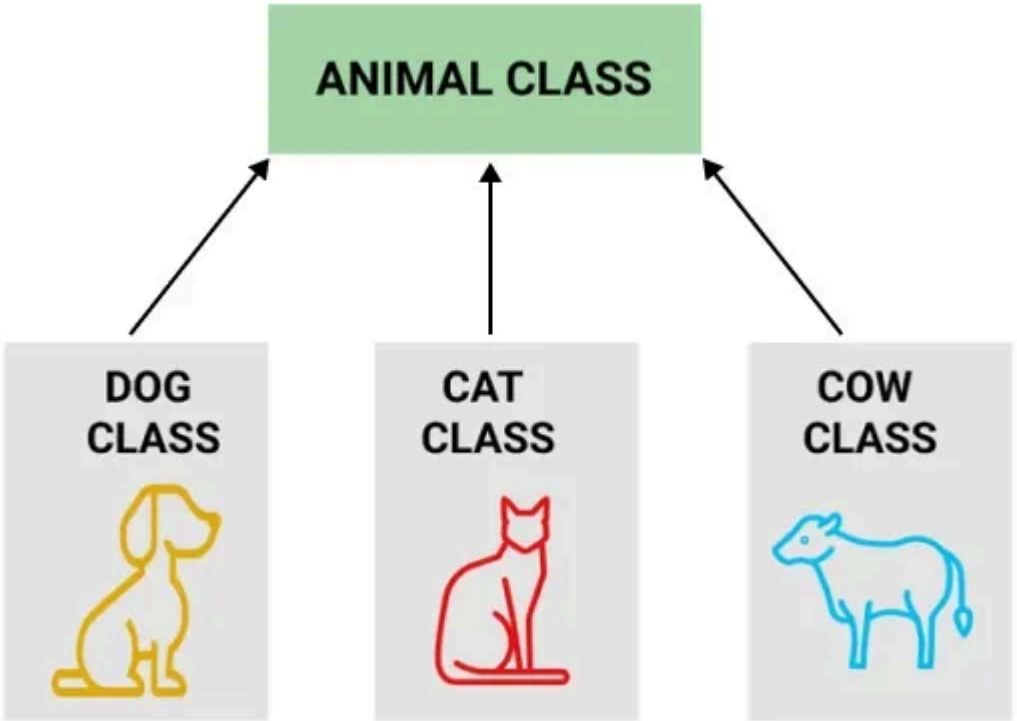
Object Inheritance is an Object-Oriented Programming (OOP) concept where one object acquires the properties (data members) and behaviors (member functions) of another object through its class.

More accurately, inheritance occurs between classes, and then objects created from the derived class inherit the features of the base class.

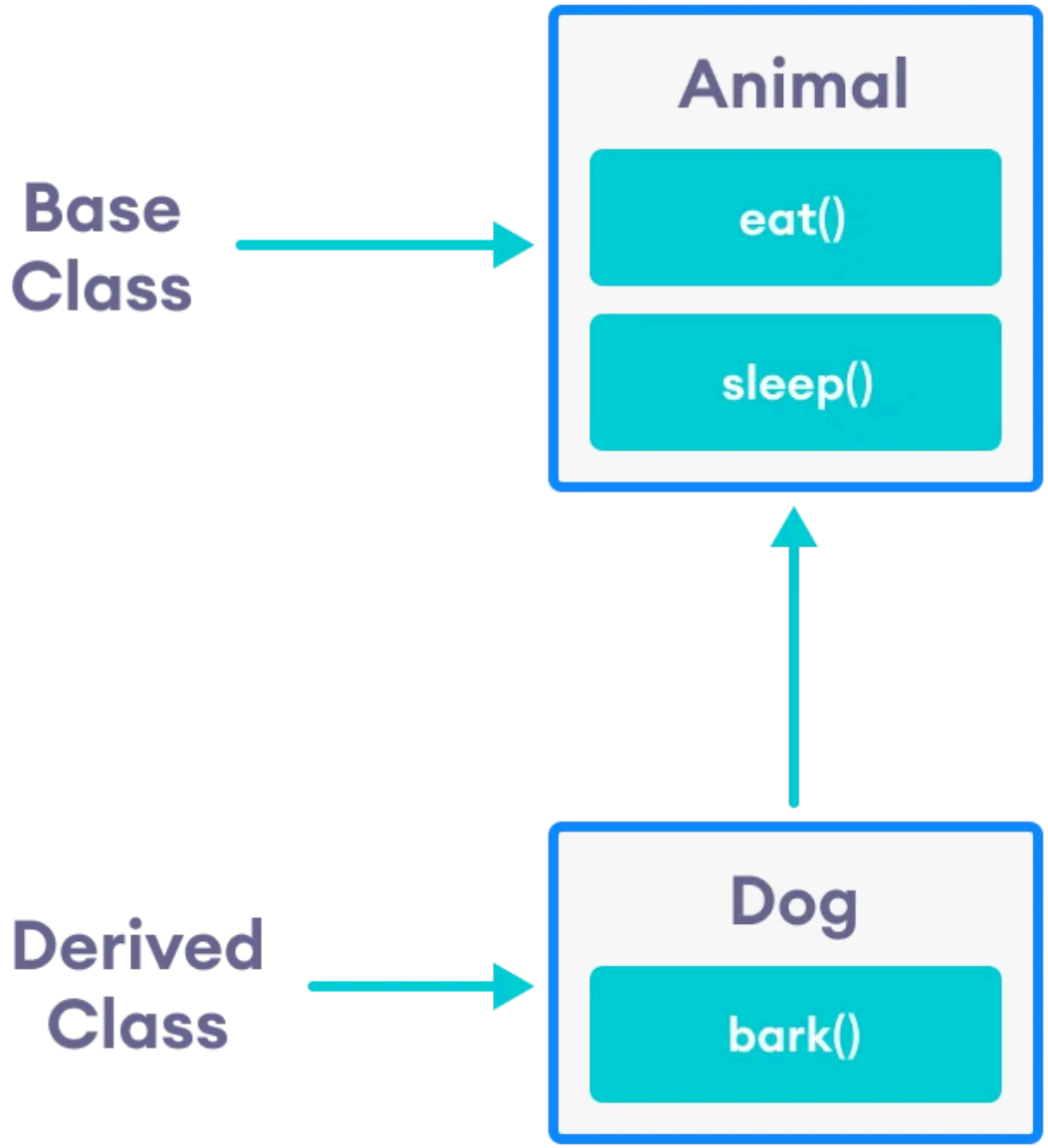
Inheritance is the process by which one class (called the derived/child class) acquires the properties and methods of another class (called the base/parent class).

Object Inheritance in JavaScript is the mechanism by which **one object can access the properties and methods of another object**. Unlike languages such as C++ or Java, JavaScript uses **Prototype-based Inheritance** instead of class-based inheritance.

Prototype-based inheritance is a feature of JavaScript where an object inherits properties and methods from another object through its **prototype**.



By Ved Vyas



,

By Ved Vyas

Code reusability (avoid writing the same code again)
Easier maintenance
Extensibility (add new features without modifying existing code)
Supports polymorphism

```
// Parent Constructor
var Vehicle = function(brand) {
    this.brand = brand;
};

// Parent method
Vehicle.prototype.start = function() {
    console.log(this.brand + " vehicle started.");
};

// Child Constructor
var Car = function(brand, model) {
    Vehicle.call(this, brand); // Inherit properties
    this.model = model;
};

// Inherit prototype methods
Car.prototype = Object.create(Vehicle.prototype);
Car.prototype.constructor = Car;

// Child method
Car.prototype.drive = function() {
    console.log(this.brand + " " + this.model + " is
driving.");
};

// Create object
var myCar = new Car("Toyota", "Fortuner");

// Call methods
myCar.start();   // Inherited method
myCar.drive();   // Child method
```

By Ved Vyas

Toyota vehicle started.
Toyota Fortuner is driving.

`Vehicle` → Parent constructor
`Car` → Child constructor
`Vehicle.call(this, brand)` → Inherits parent properties
`Object.create(Vehicle.prototype)` → Inherits parent methods
`Car.prototype.constructor = Car` → Restores the correct constructor reference
`myCar` can access both the inherited `start()` method and its own `drive()` method.

```
var Creature = function(nickname, age, x, y) {
    this.nickname = nickname;
    this.age = age + "yrs old";
    this.x = x;
    this.y = y;
};

Creature.prototype.talk = function() {
    text("SUPPP!?!?!?!??!", this.x+20, this.y+140);
};

var Hopper = function(nickname, age, x, y) {
    Creature.call(this, nickname, age, x, y);
};

Hopper.prototype = Object.create(Creature.prototype);

Hopper.prototype.draw = function() {
    fill(217, 90, 0);
    var img = getImage("creatures/Hopper-Happy");
    image(img, this.x, this.y);
    var txt = this.nickname + ", " + this.age;
    text(txt, this.x+10, this.y-7);
};
```

By Ved Vyas

```
25
26  Hopper.prototype.hooray = function() {
27     text("Hooooray!!!", this.x+29, this.y+140);
28  };
29
30  var Winston = function(nickname, age, x, y) {
31      Creature.call(this, nickname, age, x, y);
32  };
33
34  Winston.prototype = Object.create(Creature.prototype);
35
36  Winston.prototype.draw = function() {
37      fill(255, 0, 0);
38      var img = getImage("creatures/Winston");
39      image(img, this.x, this.y);
40      var txt = this.nickname + ", " + this.age;
41      text(txt, this.x+20, this.y-10);
42  };
43
```

```
45  var winstonTeen = new Winston("Winsteen", 15, 20, 50);
46  var winstonAdult = new Winston("Mr. Winst-a-lot", 30, 229, 50);
47
48  winstonTeen.draw();
49  winstonTeen.talk();
50  winstonAdult.draw();
51
52  var lilHopper = new Hopper("Little Hopper", 15, 20, 240);
53  lilHopper.draw();
54  lilHopper.talk();
55  var bigHopper = new Hopper("Big Hopper", 30, 220, 240);
56  bigHopper.draw();
57  bigHopper.hooray();
```